

MODULE 2: OPERATING SYSTEMS PRINCIPLES
WEEK 2

Learning Outcomes:

After completing this module you are expected to understand the following:

1. Structuring methods (monolithic, layered, modular, micro-kernel models)
2. Abstractions, processes, and resources
3. Concepts of application program interfaces (APIs)

A. Engage



Figure 2.0: Structured of Operating System.

B. Explain

A structure of an Operating System determines how it has been designed and how it functions. There are numerous ways of designing a new structure of an Operating system. In this post, we will learn about six combinations that have been tested and tried. These six combinations are **monolithic systems, layered systems, microkernels, client-server models, virtual machines, and exokernels**.

C. Explore

Video Titles:

1. OS - Operating System Structure | Operating Structure

YouTube Links:

1. <https://www.youtube.com/watch?v=AbNtbucMLAQ>

Module Video Filenames:

1. WEEK02_Operating System Structure.mp4

D. Elaborate

Introduction

1. Structuring methods (monolithic, layered, modular, micro-kernel models)

Monolithic System structure in an Operating System

In this organizational structure, the entire operating system runs as a single program in the kernel mode. An operating system is a collection of various procedures linked together in a binary file. In this system, any procedure can call any other procedure. Since it is running in kernel mode itself, it has all the permissions to call whatever it wants.

In terms of information hiding, there is none. All procedures are running in kernel mode, so they have access to all modules and packages of other procedures.

However, using this approach without any restrictions can lead to thousands of procedure calls, and this can lead to a messy system. For this purpose, the actual OS is constructed in a hierarchy. All the individual procedures are compiled into a single executable file using the system linker.

Even a monolithic system has a structure in which it can run in user mode. There already is a basic structure given by the organization

- 1. The main procedure that invokes the requested service procedures.
- 2. A set of service procedures that carry out system calls.
- 3. A set of utility procedures that help out the system procedures.

Layered Systems Structure in Operating Systems

As the name suggests, this system works in layers. It was designed by E.W. Dijkstra in 1968, along with some help from his students. This system was first implemented in THE system built at the Technische Hogeschool Eindhoven in the Netherlands. The THE system was a simple batch system for a Dutch computer, the Extralogica X8.

There are six layers in the system, each with different purposes.

Layer	Function
5	The operator
4	User Programs
3	Input/Output Management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

Layer 0 – Processor Allocation and Multiprogramming – This layer deals with the allocation of processor, switching between the processes when interrupts occur or when the timers expire.

The sequential processes can be programmed individually without having to worry about other processes running on the processor. That is, layer 0 provides that basic multiprogramming of the CPU

Layer 1 – Memory and Drum Management – This layer deals with allocating memory to the processes in the main memory. The drum is used to hold parts of the processes (pages) for which space couldn't be provided in the main memory. The processes don't have to worry if there is available memory or not as layer 1 software takes care of adding pages wherever necessary.

Layer 2 – Operator-Process communication – In this layer, each process communicates with the operator (user) through the console. Each process has its own operator console and can directly communicate with the operator.

Layer 3 – Input/Output Management – This layer handles and manages all the I/O devices, and it buffers the information streams that are made available to it. Each process can communicate directly with the abstract I/O devices with all of its properties.

Layer 4 – User Programs – The programs used by the user are operated in this layer, and they don't have to worry about I/O management, operator/processes communication, memory management, or the processor allocation.

Layer 5 – The Operator – The system operator process is located in the outer most layer.

Microkernels system in an operating system

Traditionally, all the layers of the OS in a layered system went into the kernel. So they all had root access to the OS, and any small bug in any layer could be fatal to the OS.

Famous examples of a microkernel system include Integrity, K42, PikeOS, Symbian, and MINIX 3

The primary purpose of this system is to provide high reliability. Because of the high reliability that it provides, the applications of microkernels can be seen in real-time, industrial, avionics (electronics fitted in aircraft and aviation), and military applications that are mission-critical and require high reliability.

2. Abstractions, processes, and resources

The two main purposes of an operating system are to provide an interface between the hardware and the application programs and to manage the various pieces that make up a computer. To be more precise, these pieces are called *resources*. A resource is "any object which can be allocated within a system". Some examples of resources are processors (CPUs), input/output devices, files, and memory (RAM). Thus, we can restate the purpose of the operating system in terms of resources. The operating system manages resources (resource allocation) and provides an interface to resources for application programs (resource abstraction).

In addition to the responsibility of resource allocation, the operating system is also responsible for resource abstraction. Resource abstraction is the process of "hiding the details of how the hardware operates, thereby making computer hardware relatively easy for an application programmer to use". One way in which the operating system might implement resource abstraction is to provide a single abstract disk interface which will be the same for both the hard disk and floppy disk. Such an abstraction saves the programmer from needing to learn the details of both hardware interfaces. Instead, the programmer only needs to learn the disk abstraction provided by the operating system.

3. Concepts of application program interfaces (APIs)

An API can specify the interface between an application and the operating system. POSIX, for example, specifies a set of common APIs that aim to enable an application written for a POSIX conformant operating system to be compiled for another POSIX conformant operating system.

Linux and Berkeley Software Distribution are examples of operating systems that implement the POSIX APIs.

Microsoft has shown a strong commitment to a backward-compatible API, particularly within its Windows API (Win32) library, so older applications may run on newer versions of Windows using an executable-specific setting called "Compatibility Mode".

An API differs from an application binary interface (ABI) in that an API is source code based while an ABI is binary based. For instance, POSIX provides APIs while the Linux Standard Base provides an ABI.

E. Evaluate

ASSESSMENT:

Instructions: You may write your answer on the Answer Sheet (AS) provided on this module.

I. Identification (2-Point Each):

1. In this layer, each process communicates with the operator (user) through the console.
2. This layer handles and manages all the I/O devices, and it buffers the information streams that are made available to it.
3. This layer deals with the allocation of processor, switching between the processes when interrupts occur or when the timers expire.
4. This layer deals with allocating memory to the processes in the main memory.
5. In this organizational structure, the entire operating system runs as a single program in the kernel mode.
6. This system was first implemented in THE system built at the Technische Hogeschool Eindhoven in the Netherlands.
7. The programs used by the user are operated in this layer, and they don't have to worry about I/O management, operator/processes communication, memory management, or the processor allocation.
8. The system operator process is located in the outer most layer.
9. Traditionally all the layers of the OS in a layered system went into the _____.
10. It can specify the interface between an application and the operating system.

Laboratory Activity

Task 2.1

Follow these instructions carefully. Do **NOT** just continue without completely reading each step of the instructions carefully or you will probably not install the software correctly. If in doubt ask a classmate or the instructor to verify what you are doing. If you cannot create a project after installing, then you will need to reinstall following these instructions more carefully.

1. Download the codeblocks-20.03mingw-setup.exe binary for your computer from www.codeblocks.org/downloads/binaries.

The **mingw-setup** version includes the gcc/g++ compiler preconfigured and makes our installation easier.

2. Run the setup file and press the Next > button after reading the dialog.

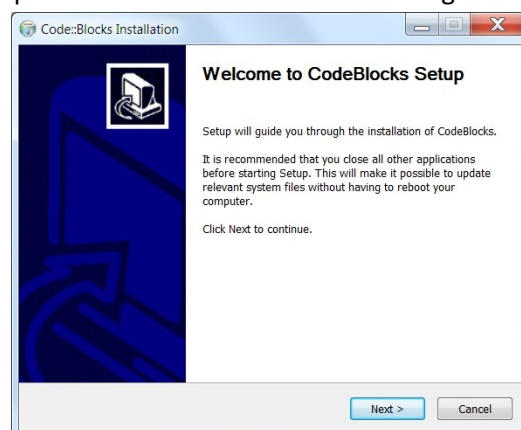


Figure 2.1

3. Read the license and press the I Agree button.



Figure 2.2.

4. Choose the default components as shown and press the Next > button.

For Windows, if you do not see **minGW** go back to step 1 and download the correct file.

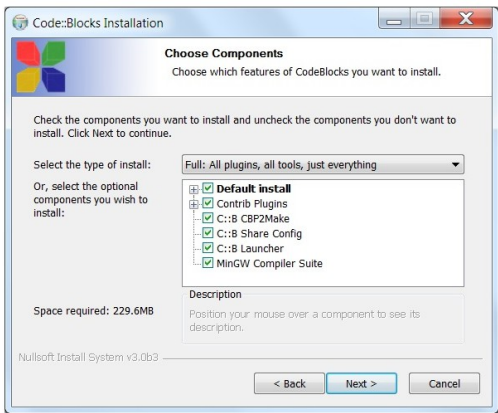


Figure 2.3.

5. Choose an installation folder (C:\CodeBlocks) as shown below and press the Install button.

Do NOT use the default location or you may have problems later in the installation of SFML or the creation of a project. Instead, install at C:\CodeBlocks to make working at the command line and with SFML easier.

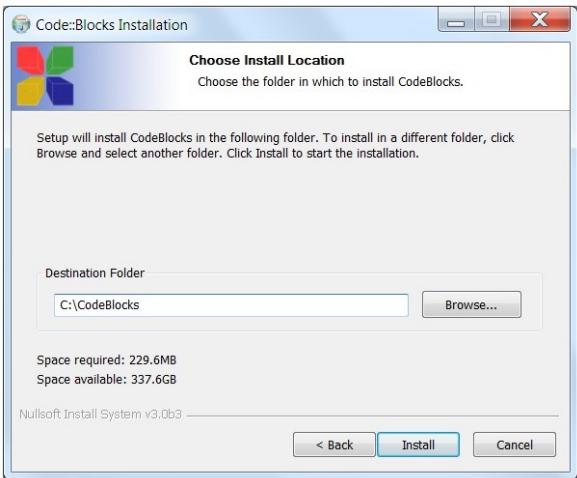


Figure 2.4.

6. Choose to run Code:Blocks now by pressing the Yes button.
This step allows us to immediately configure the compiler. You can come back to this step if you decide to run Code::Blocks later.

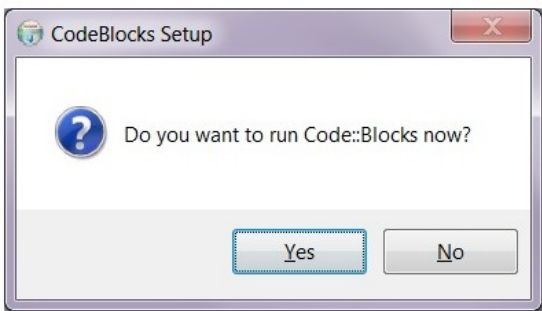


Figure 2.5.

7. Select the GNU GCC Compiler as shown, press the Set as default button and then press the OK button.
If you have previously installed Code::Blocks, you may not see this dialog. If you are changing the compiler, you will need to manually set the compiler path (Settings -> Compiler -> Toolchain executables -> Compiler's installation directory).

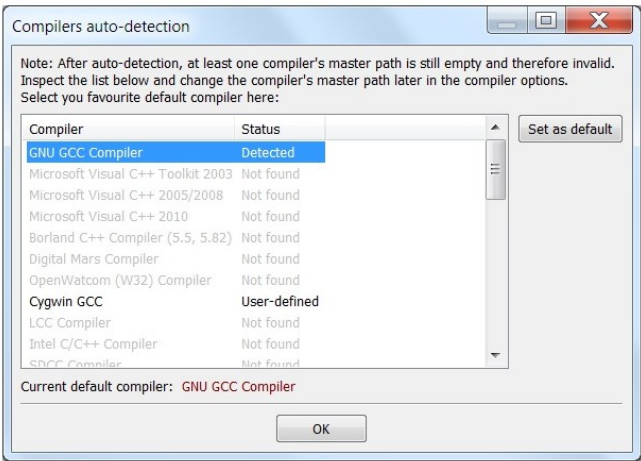


Figure 2.6.

8. Make a choice for the file association and press the OK button.
Choosing to associate Code::Blocks with C++ files means that Code::Blocks will start automatically whenever a C++ file is selected, such as by double-clicking an icon on Windows. If unsure, select "No, leave everything as it is (but ask me again next time)".

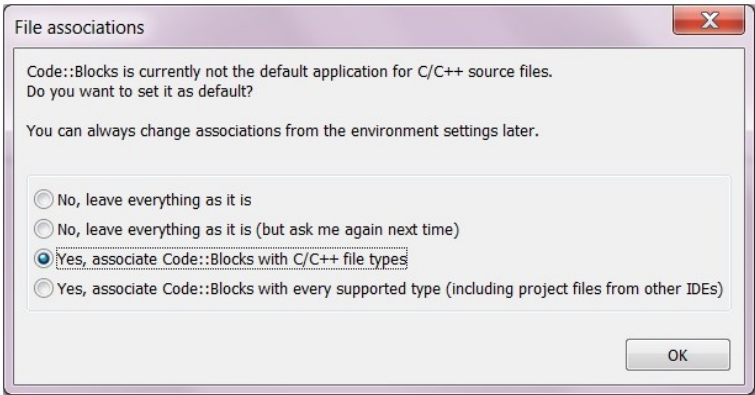


Figure 2.7.

9. Exit Code::Blocks (File -> Quit) and then press the Finish button to exit the installer.

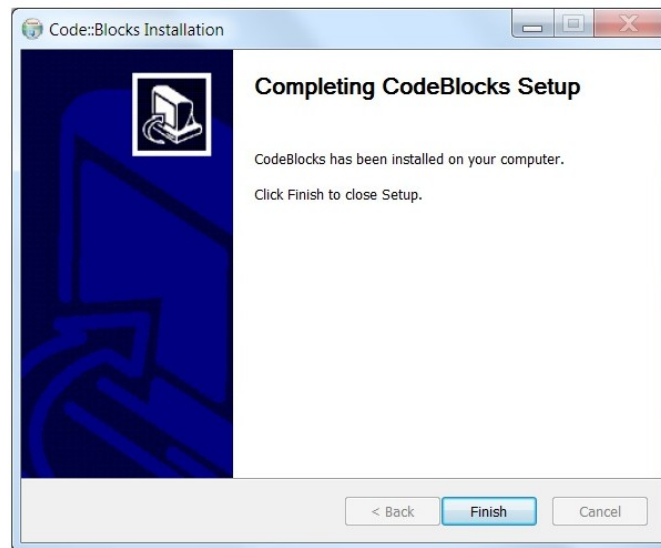


Figure 2.8.

10. Test the installation by creating a project as shown in the next section.

2.2 Writing Toy Programs (without Creating a Project)

To write toy programs (such as few-line simple programming exercises):

1. File ⇒ New ⇒ **Empty File**.
2. Enter (copy and paste) the following codes:

```
// First C++ program to say Hello

#include <iostream>

using namespace std;

int main() {

    cout << "Hello, world!" << endl;

    return 0;

}
```

Save the file as "Hello.cpp" in your project directory (e.g., "d:\project").

3. Build (Compile and Link): Select "Build" menu ⇒ Build (Ctrl-F9).
4. Run: Select "Build" menu ⇒ Run (Ctrl-F10).

The drawback is you cannot debug program without creating a project.

References:

- 1. What are the different structures of an Operating System?–
<https://technobyte.org/different-structures-of-operating-system/>
- 2. API–
https://en.wikipedia.org/wiki/API#Operating_systems

Facilitated By:		
Name	:	
MS Teams Account (email)	:	
Facebook Page	:	